

Branchless Programming

As we established in [the previous section](#), branches that can't be effectively predicted by the CPU are expensive as they may cause a long pipeline stall to fetch new instructions after a branch mispredict. In this section, we discuss the means of removing branches in the first place.

#Predication

We are going to continue the same case study we've started before — we create an array of random numbers and sum up all its elements below 50:

```
for (int i = 0; i < N; i++)
    a[i] = rand() % 100;

volatile int s;

for (int i = 0; i < N; i++)
    if (a[i] < 50)
        s += a[i];
```

Our goal is to eliminate the branch caused by the `if` statement. We can try to get rid of it like this:

```
for (int i = 0; i < N; i++)
    s += (a[i] < 50) * a[i];
```

The loop now takes ~7 cycles per element instead of the original ~14. Also, the performance remains constant if we change `50` to some other threshold, so it doesn't depend on the branch probability.

But wait... shouldn't there still be a branch? How does `(a[i] < 50)` map to assembly?

There are no Boolean types in assembly, nor any instructions that yield either one or zero based on the result of the comparison, but we can compute it indirectly like this: `(a[i] - 50) >> 31`. This trick relies on the [binary representation of integers](#), specifically on the fact that if the expression `a[i] - 50` is negative (implying `a[i] < 50`), then the highest bit of the result will be set to one, which we can then extract using a right-shift.

```

mov  ebx, eax    ; t = x
sub  ebx, 50     ; t -= 50
sar  ebx, 31     ; t >>= 31
imul eax, ebx    ; x *= t

```

Another, more complicated way to implement this whole sequence is to convert this sign bit into a mask and then use bitwise `and` instead of multiplication: `((a[i] - 50) >> 31 - 1) & a[i]`. This makes the whole sequence one cycle faster, considering that, unlike other instructions, `imul` takes 3 cycles:

```

mov  ebx, eax    ; t = x
sub  ebx, 50     ; t -= 50
sar  ebx, 31     ; t >>= 31
; imul eax, ebx ; x *= t
sub  ebx, 1      ; t -= 1 (causing underflow if t = 0)
and  eax, ebx    ; x &= t

```

Note that this optimization is not technically correct from the compiler's perspective: for the 50 lowest representable integers – those in the $[-2^{31}, -2^{31}+49]$ range – the result will be wrong due to underflow. We know that all numbers are all between 0 and 100, and this won't happen, but the compiler doesn't.

But the compiler actually elects to do something different. Instead of going with this arithmetic trick, it used a special `cmov` (“conditional move”) instruction that assigns a value based on a condition (which is computed and checked using the flags register, the same way as for jumps):

```

mov    ebx, 0      ; cmov doesn't support immediate values, so we need a zero
register
cmp    eax, 50
cmovge eax, ebx    ; eax = (eax ≥ 50 ? eax : ebx=0)

```

So the code above is actually closer to using a ternary operator like this:

```

for (int i = 0; i < N; i++)
    s += (a[i] < 50 ? a[i] : 0);

```

Both variants are optimized by the compiler and produce the following assembly:

```

mov     eax, 0
mov     ecx, -4000000
loop:
mov     esi, dword ptr [rdx + a + 4000000] ; load a[i]
cmp     esi, 50
cmovge  esi, eax                          ; esi = (esi ≥ 50 ? esi : eax=0)
add     dword ptr [rsp + 12], esi          ; s += esi
add     rdx, 4
jnz     loop                             ; "iterate while rdx is not zero"

```

This general technique is called *predication*, and it is roughly equivalent to this algebraic trick:

$$x = c \cdot a + (1 - c) \cdot b$$

This way you can eliminate branching, but this comes at the cost of evaluating *both* branches and the `cmov` itself. Because evaluating the “ $\geq$ ” branch costs nothing, the performance is exactly equal to the “always yes” case in the branchy version.

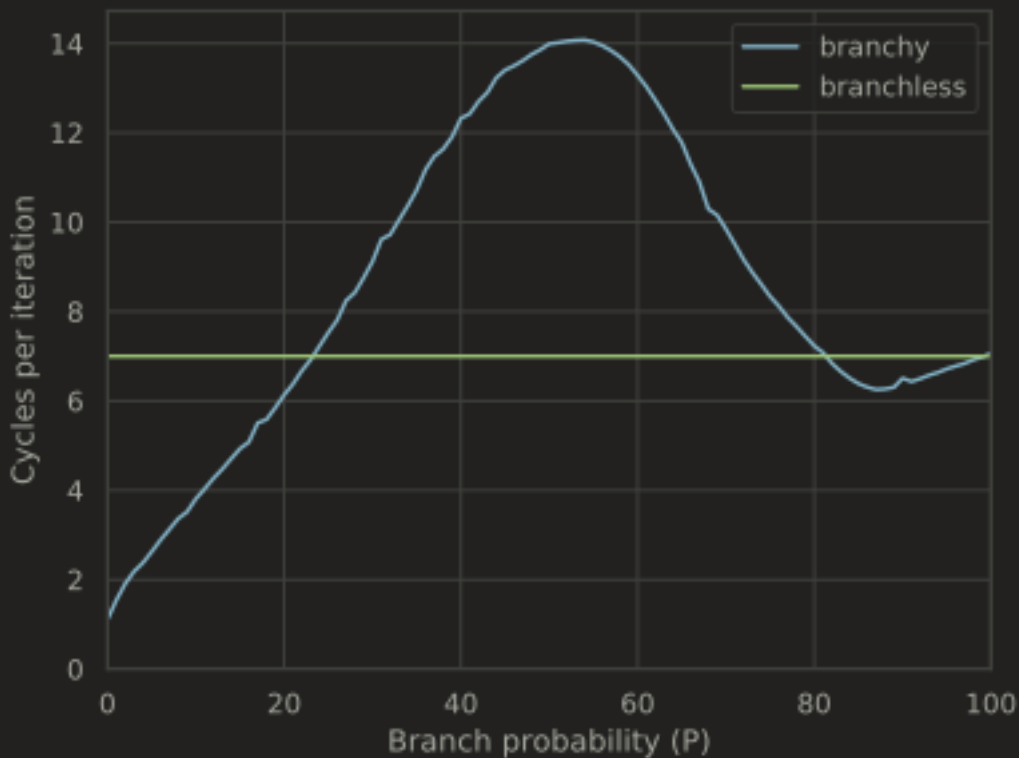
#When Predication Is Beneficial

Using predication eliminates a control hazard but introduces a data hazard. There is still a pipeline stall, but it is a cheaper one: you only need to wait for `cmov` to be resolved and not flush the entire pipeline in case of a mispredict.

However, there are many situations when it is more efficient to leave branchy code as it is. This is the case when the cost of computing *both* branches instead of just *one* outweighs the penalty for the potential branch mispredictions.

In our example, the branchy code wins when the branch can be predicted with a probability of more than ~75%.

```
for (int i = 0; i < N; i++) if (a[i] < P) s += a[i]
```



This 75% threshold is commonly used by the compilers as a heuristic for determining whether to use the `cmov` or not. Unfortunately, this probability is usually unknown at the compile time, so it needs to be provided in one of several ways:

- We can use `profile-guided optimization` which will decide for itself whether to use predication or not.
- We can use `likeliness attributes` and `compiler-specific intrinsics` to hint at the likeliness of branches: `__builtin_expect_with_probability` in GCC and `__builtin_unpredictable` in Clang.
- We can rewrite branchy code using the ternary operator or various arithmetic tricks, which acts as sort of an implicit contract between programmers and compilers: if the programmer wrote the code this way, then it was probably meant to be branchless.

The “right way” is to use branching hints, but unfortunately, the support for them is lacking. Right now these hints seem to be lost by the time the compiler back-end decides whether a `cmov` is more beneficial. There is some progress towards making it possible, but currently, there is no good way of forcing the compiler to generate branch-free code, so sometimes the best hope is to just write a small snippet in assembly.

#Larger Examples

Strings. Oversimplifying things, an `std::string` is comprised of a pointer to a null-terminated `char` array (also known as a “C-string”) allocated somewhere on the heap and one integer containing the string size.

A common value for a string is the empty string – which is also its default value. You also need to handle them somehow, and the idiomatic approach is to assign `nullptr` as the pointer and `0` as the string size, and then check if the pointer is null or if the size is zero at the beginning of every procedure involving strings.

However, this requires a separate branch, which is costly (unless the majority of strings are either empty or non-empty). To remove the check and thus also the branch, we can allocate a “zero C-string,” which is just a zero byte allocated somewhere, and then simply point all empty strings there. Now all string operations with empty strings have to read this useless zero byte, but this is still much cheaper than a branch misprediction.

Binary search. The standard binary search [can be implemented](#) without branches, and on small arrays (that fit into cache) it works ~4x faster than the branchy

`std::lower_bound` :

```
int lower_bound(int x) {
    int *base = t, len = n;
    while (len > 1) {
        int half = len / 2;
        base += (base[half - 1] < x) * half; // will be replaced with a "cmov"
        len -= half;
    }
    return *base;
}
```

Other than being more complex, it has another slight drawback in that it potentially does more comparisons (constant $\lceil \log_2 n \rceil$ instead of either $\lfloor \log_2 n \rfloor$ or $\lceil \log_2 n \rceil$) and can't speculate on future memory reads (which acts as prefetching, so it loses on very large arrays).

In general, data structures are made branchless by implicitly or explicitly *padding* them so that their operations take a constant number of iterations. Refer to [the article](#) for more complex examples.

Data-parallel programming. Branchless programming is very important for SIMD applications because they don't have branching in the first place.

In our array sum example, removing the `volatile` type qualifier from the accumulator allows the compiler to `vectorize` the loop:

```
/* volatile */ int s = 0;

for (int i = 0; i < N; i++)
    if (a[i] < 50)
        s += a[i];
```

It now works in ~0.3 per element, which is mainly bottlenecked by the memory.

The compiler is usually able to vectorize any loop that doesn't have branches or dependencies between the iterations – and some specific small deviations from that, such as `reductions` or simple loops that contain just one if-without-else. Vectorization of anything more complex is a very nontrivial problem, which may involve various techniques such as `masking` and `in-register permutations`.